# Neural error-correction trees

ELL409 | Tree type network revisited | Final report | 11 October 2026

Repository: https://github.com/Harshal-Narsaria/ELL409

We built image classifiers whose tree nodes are small neural networks, using CIFAR-10 and Cats vs Dogs. The work covers four feature strategies, near-perfect training fit, double-descent experiments, node interpretation and model-size reduction.

Our main question became: how much learning comes from the image features, the root classifier, and the correction children? A strong CNN made the first model successful but hid much of the tree contribution. We therefore tried weaker features, fixed roots, a fitting repair, and controlled training-label noise.

| Assignment requirement | What we completed |
|---|---|
| Provide suitable features | Shared trained CNN, raw pixels/local CNNs, CNN stopped near 80%, and frozen random CNN features. |
| Overfit the training set | Original-label trees reach 99.9933% on CIFAR and 100% on Cats. Random-feature trees also reach 100% on both. |
| Try to observe double descent | Width, depth and epoch studies; a 64-configuration noise/control study; 48 longer-budget follow-up configurations. |
| Interpret and reduce the model | Response examples, Grad-CAM, occlusion, narrower nodes, subtree pruning and compact exports. |

| Original shared-CNN tree | CIFAR-10 | Cats vs Dogs |
|---|---|---|
| Training accuracy | 99.9933% (3 errors) | 100% (0 errors) |
| Validation accuracy | 80.48% | 90.54% |
| Test accuracy | 80.02% | 89.94% |
| Deployed parameters | 486,784 | 336,952 |

All noise follow-up configurations and their test evaluations are complete. Noisy CIFAR shows a suggestive double-descent shape and repeatable improvement beyond interpolation. In the longer runs, that improvement comes from wider root MLPs with inactive correction children; it is not evidence that tree growth itself caused double descent.

The report focuses on the reasons for the experiments and the evidence. The repository archives retain unsuccessful attempts, detailed tables, all graph sets, code, configurations and selected models.

# 1. Model and experimental setup

Each node receives image features and produces a binary score. Child A learns when a positive correction is needed; child B learns when a negative correction is needed. Their decisions feed upward into the parent. Difficult child tasks can receive further correction children [1].

Corrected score = base score + alpha x A - beta x B + bias adjustment. Alpha and beta are nonnegative. A score of zero or above predicts class 1.

| Parent prediction | Correct label | A target | B target |
|---|---|---|---|
| 0 | 0 | 0 | Ignore |
| 1 | 1 | Ignore | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |

Ignore means that example is excluded from the child loss. We use actual classification mistakes to construct these tasks. Nodes train with binary cross-entropy; parent correction strengths are fitted after child decisions are frozen.

The usual shared-feature node is 128 inputs -> hidden ReLU units -> one score. Cats vs Dogs uses one binary tree. CIFAR uses ten one-versus-rest trees; the largest corrected root score chooses the final class.

| Dataset | Training | Validation | Test |
|---|---|---|---|
| CIFAR-10 | 45000 | 5000 | 10000 |
| Cats vs Dogs | 18608 | 2326 | 2326 |

CIFAR uses its official test set [2]. Cats was prepared with TensorFlow Datasets, which filters corrupt images [3]; one conflicting duplicate pair was removed. Duplicates stay within one split, and normalization uses training data only. Split manifests are saved.

CIFAR images are 32 x 32. Cats images are 128 x 128 in the original model and 32 x 32 in later studies. Training uses no stochastic augmentation, dropout, label smoothing or weight decay. Added label noise is a separate study; original-label results are preserved.

Experiments ran in PyTorch on an RTX 2050 with 4 GB GPU memory. Cached features save repeated CNN computation. Checkpoints retain optimizer and random-generator states, so interrupted training can resume.

Validation guided model selection. Later studies reused the same test partitions and are exploratory. Test labels did not train model weights. We retained the original partitions for comparisons and did not reserve another independent holdout before exploring further. This limits the independence of the later test evidence. Some comparisons also change resolution or architecture.

# 2. Implementation and training settings

All learned networks use Adam. Image classifiers use cross-entropy; binary node tasks use class-balanced binary cross-entropy. Checkpoints are chosen first by the fewest training errors, then by lower unweighted loss. The settings below come from the saved run configurations.

| Stage | Learning rate / batch | Training budget and capacity |
|---|---|---|
| V1 shared CNN | 0.001 / CIFAR 64; Cats 16 | 30 epochs; channels 32/64/128, two 3 x 3 convolutions and pooling per block; global pooling to 128 features. |
| V1 correction tree | 0.001 / 1,024 features | Width 64; 100 epochs per node; maximum depth 4 and 31 nodes per binary tree. Long width sweeps use 300 epochs. |
| V2 pixels / local CNNs | 0.001 / MLP 512; CNN 128 | Root width 4, 120 epochs. Main correction width 16; MLP 120 / CNN 60 epochs. Extended fits: width 32, MLP 240 / CNN 90 epochs. |
| V3 CNN80 + children | 0.001 / CNN 64; MLP 1,024 | CNN stops at first full-training evaluation >= 80%; children train for 200 epochs. Widths 32/64/128/256; original depth grid 0-10, width-32 follow-up to 20. |
| V4 random CNN + tree | 0.001 / 1,024 features | Random CNN channels 16/32/64 and 2 x 2 spatial pooling: 256 features. Frozen width-8 root; 200 epochs per learned node. |
| V5 clean/noisy capacity study | 0.001 / 1,024 features | Frozen CNN80 features; depth 1, three allocated nodes per binary tree. 400 epochs, then a separate common 1,200-epoch follow-up. |

Growth rule: train a node, construct masked A/B tasks from its errors, and add a child only when it has positive correction targets and there is room under the depth/node cap. V1 stops growing a branch when it has no base errors. The 99.5% near-fit threshold is a reporting target, not an early stop that replaces the configured epoch budget.

Parent fitting holds the base scores and child decisions fixed. V1 fits alpha, beta and the bias with full-batch Adam at learning rate 0.05 for 150 epochs, projecting alpha and beta to nonnegative values. It also tries a SciPy HiGHS linear program that minimizes alpha + beta subject to a signed margin of at least one. A worse refit is rejected using training errors, then loss.

The later positive-margin repair is described with its paired comparison on page 6. V4/V5 use that verified repair when feasible, with the original fit as fallback. No held-out labels enter parent fitting. Seeds are 409 for primary runs and 410/411 for repeats; V5 corruption seed 150409 is fixed across widths.

No augmentation, dropout, label smoothing or weight decay is used. Matching interrupted tasks restore optimizer and random states; changed tasks train afresh. Detailed configurations and histories are archived.

# 3. Feature choices: why we changed the design

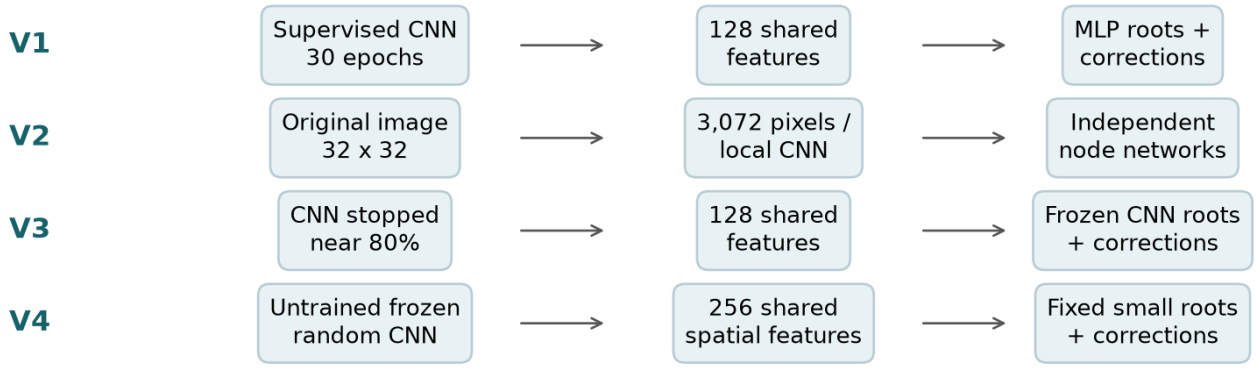



Figure 1. Four implemented ways to supply features to the correction tree.

| Approach | Reason and observation |
|---|---|
| V1: trained shared CNN | Train one compact CNN with 32/64/128 channels and global pooling, freeze its 128 features, then train the tree. Efficient and accurate, but root MLPs already fit almost perfectly. |
| V2: pixels and local CNNs | Use pixel MLP nodes; a hybrid trains independent small CNNs at the first correction level. A fixed width-4 root leaves learning to the children. Local CNNs improve validation over pixels, but these fits miss 99.5% training accuracy. |
| V3: CNN stopped near 80% | Freeze the CNN at its first crossing of 80% training accuracy and keep its root decisions. The tree raises training accuracy, exposing a parent-fitting bottleneck. |
| V4: frozen random CNN | Use an untrained CNN to supply 256 spatial features and freeze a trained width-8 root. Correction nodes must supply much more supervised learning. Both datasets reach 100% training fit, with lower test accuracy. |

| Selected result | CIFAR train / test % | Cats train / test % |
|---|---|---|
| V1: shared trained CNN tree | 99.9933 / 80.02 | 100.00 / 89.94 |
| V3: CNN80 + progressive tree | 99.29 / 76.43 | 94.53 / 79.92 |
| V4: random features + tree | 100.00 / 48.75 | 100.00 / 68.10 |

V3 froze the CNN at epoch 6 on CIFAR (80.14% training accuracy) and epoch 5 on Cats (83.68%). V4 used no supervised CNN training. V2 used separate CNNs only at selected nodes; we did not run a CNN at every node in every version.

The contribution is the experimental progression: deliberately reduce how much the trunk or root can solve, then measure the corrections. These are adaptations of established methods, not a claim to have invented a new general learning algorithm.

# 4. Baselines: does the tree add value?

The V1 CNN-only baseline uses the trained CNN with its conventional classification head. The root-MLP control trains without correction children on the same cached features. These are separate trained models; the pruned model later provides a direct removal check.

| V1 dataset | Classifier | Train % | Val % | Test % |
|---|---|---|---|---|
| CIFAR | CNN only | 98.5644 | 79.54 | 79.08 |
| CIFAR | Root MLPs | 99.9800 | 80.44 | 80.14 |
| CIFAR | Correction tree | 99.9933 | 80.48 | 80.02 |
| Cats | CNN only | 99.5002 | 89.68 | 89.12 |
| Cats | Root MLPs | 99.9624 | 90.50 | 89.90 |
| Cats | Correction tree | 100.0000 | 90.54 | 89.94 |

Compared with CNN-only, the tree gains 0.94 percentage points of CIFAR test accuracy and about 0.82 points on Cats. Compared with the root MLPs, CIFAR test accuracy is slightly lower (80.02% versus 80.14%) and Cats is almost unchanged (89.94% versus 89.90%). Most of V1 performance therefore comes from learned features and the root classifier.

V2 controls use 32 x 32 images and independent image-level networks. The unrestricted pixel MLP is approximately parameter-matched to the extended pixel tree. These runs have validation measurements, not final test results; they are not directly comparable with the larger V1 CNN.

| V2 dataset | Classifier | Parameters | Train % | Val % |
|---|---|---|---|---|
| CIFAR | Pixel tree | 2,680,672 | 98.73 | 45.40 |
| CIFAR | Matched pixel MLP | 2,674,420 | 100.00 | 51.98 |
| CIFAR | Local-CNN tree | 4,044,524 | 97.50 | 60.58 |
| CIFAR | Small CNN roots | 55,240 | 78.86 | 69.66 |
| Cats | Pixel tree | 209,044 | 98.43 | 63.37 |
| Cats | Matched pixel MLP | 209,036 | 100.00 | 66.12 |
| Cats | Local-CNN tree | 425,092 | 97.29 | 72.66 |
| Cats | Small CNN roots | 5,524 | 85.44 | 80.14 |

The matched pixel MLP generalizes better than the pixel tree in these runs. Local CNN corrections improve validation over pixels, but the small CNN-only controls perform better still. Thus, extra parameters and tree structure alone do not explain better generalization.

# 5. Tree contribution and parent-fitting repair

With deliberately weaker features, corrections have a much larger fitting role. The following comparisons remove children from the identical learned model; they isolate the correction contribution to training accuracy.

| Paired training comparison | Root only | With corrections |
|---|---|---|
| V4 / CIFAR | 63.36% | 100.00% |
| V4 / Cats | 76.79% | 100.00% |
| V5 noisy CIFAR / width 128, 400 epochs | 90.74% | 97.15% |

Random features expose the children's contribution, although training fit improves more than generalization. The noisy width-128 comparison shows that children can also help fit changed labels. These findings differ from the later wide models whose children become inactive.

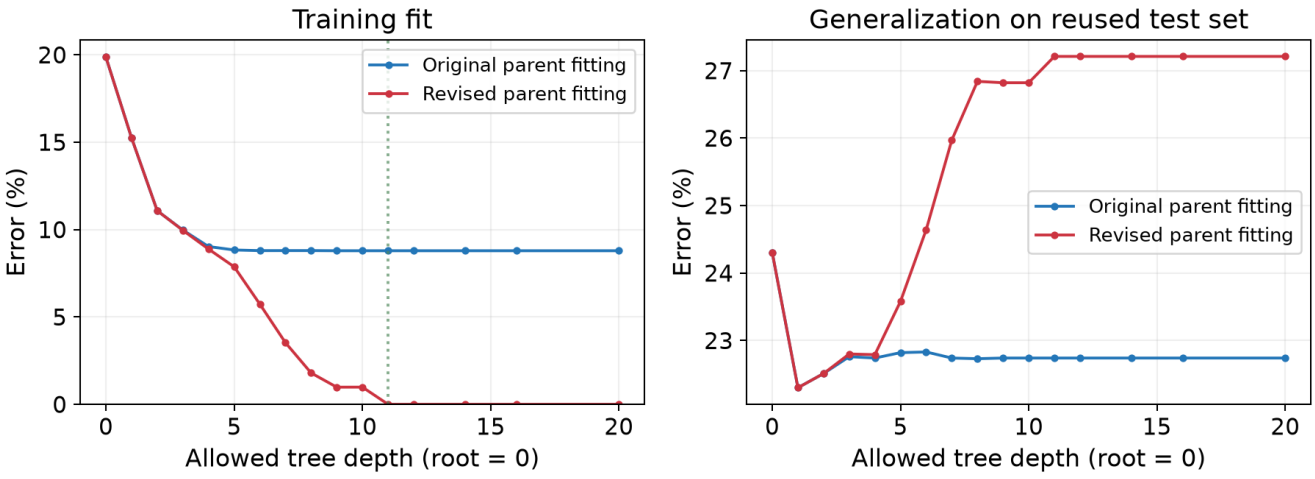



Figure 2. CIFAR width-32 follow-up, seed 409: compare parent fitting with the same learned node bases.

Our most targeted refinement was the parent-fitting repair. The original linear program required every signed corrected score to be at least one, which can be impossible even when correct classification is feasible. We kept the same base-score coefficient, correction signs and learned bases, but allowed a smaller positive margin.

The revised linear program first maximizes a common margin m between zero and one. If m > 10^-8, a second program minimizes alpha + beta while keeping at least m/2, leaving room for rounding. We then verify zero errors using the actual float32 predictions; otherwise, the original fit is used. No test or validation labels are consulted.

With the same learned bases, training accuracy improves from 91.21% to 100%; test accuracy changes from 77.26% to 72.79%. Perfect fitting first occurs at allowed depth 11. This isolates a fitting bottleneck, but it does not improve generalization.

Later saturated depths reuse the same effective model and are not independent capacity points.

# 6. Double descent with training-label noise

Double descent means held-out error falls, rises near the capacity needed to fit the training set, then falls again [4]. Training error locates interpolation: zero classification errors. Test/validation error is the main outcome; a falling training curve alone is not double descent.

Original-label width, depth and epoch studies produced hints, especially with random CNN features. The CIFAR depth-6 slice peaked at its first zero-error width, but active node counts changed and the peak was not repeated across seeds. We therefore added a separate controlled noise study.

V5 compares 0% and 15% training-label noise using the same frozen CNN80 features, splits and optimizer. Exactly 6,750 CIFAR labels and 2,792 Cats labels were changed. Validation and test labels stayed original. The CNN had learned clean labels before corruption; this tests downstream tree fitting, not end-to-end noisy CNN training.

The broad study contains 64 completed noise/control configurations across both datasets, including primary widths 2/4/8/16/32/64/128/256/512/1,024 and selected repeats. Figure 3 shows the primary seed; detailed repeat measurements are archived.

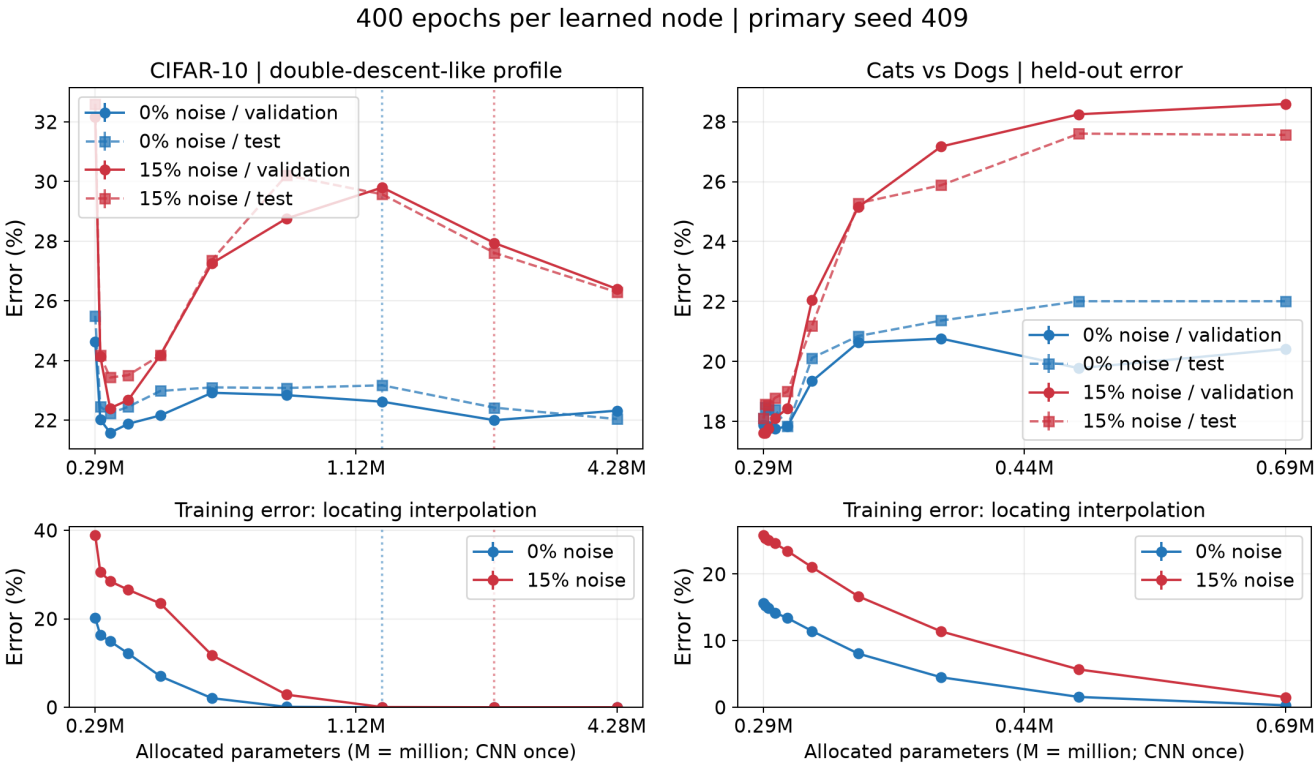



Figure 3. Double-descent investigation: 0% and 15% label noise, 400 epochs, primary seed 409. The noisy CIFAR curve falls, rises and falls again. Training error below locates interpolation; dotted lines mark the first tested zero-error width.

Noisy CIFAR test error follows 32.60% -> 23.43% -> 30.21% -> 26.29% at widths 2/8/128/1,024: a double-descent-like shape. Its peak precedes first exact interpolation at width 512. Some node losses were still falling near the end, so incomplete optimization could affect the shape. These data do not establish why the peak is displaced; we tested longer training instead of assigning a cause.

The 400-epoch noisy Cats sweep still made 281 training errors at width 1,024. It therefore had not reached interpolation, and its small late test change could not establish a second descent.

# 7. Noisy-model follow-up: after interpolation

We completed 48 configurations at a common budget of 1,200 epochs: CIFAR widths 256/512/1,024 and Cats widths 256/512/1,024/2,048/4,096, both noise levels and three optimizer seeds. The lower Cats widths were added using training diagnostics to cover below interpolation.

When a node has exactly the same training examples and targets, it resumes from its 400-epoch checkpoint. If its task changes, it trains afresh for 1,200 epochs. Each predeclared study is completed before its test evaluation. Repeats vary optimizer seeds, while the CNN and label-corruption map remain fixed.

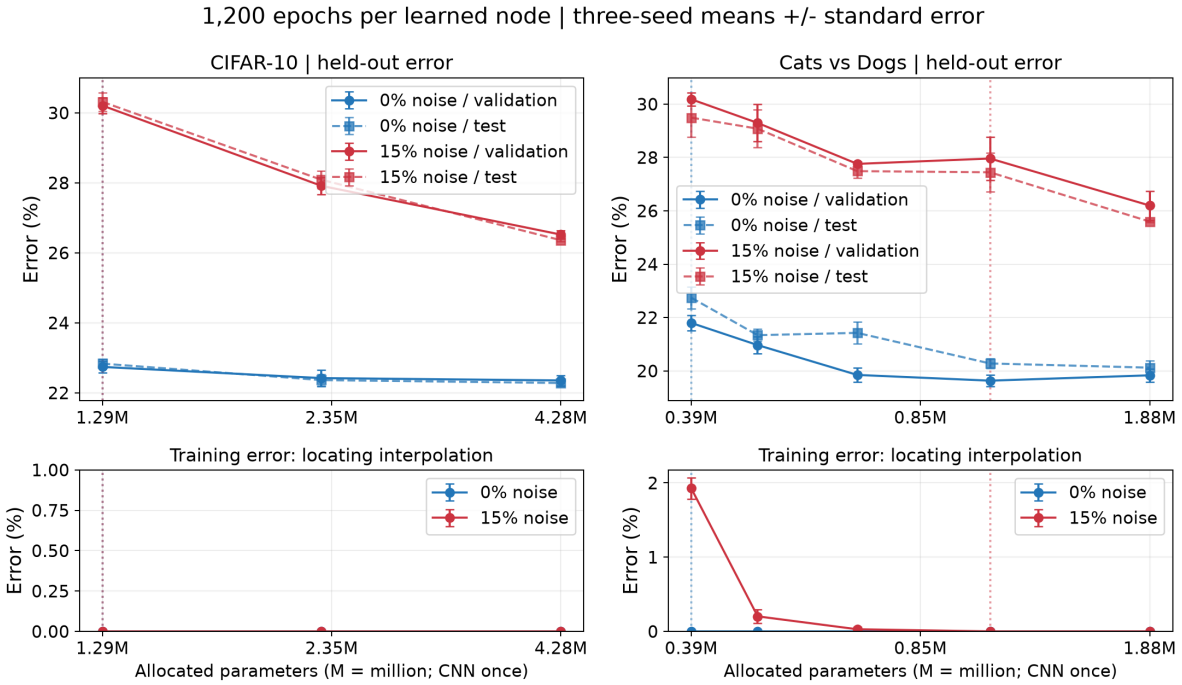



Figure 4. Mean error +/- standard error across three optimizer seeds. Dotted lines mark the first tested width with zero training errors in all three runs. Clean-label control and 15% noise share the same features.

| 15% noise | Width | Zero-error runs | Val error % | Test error % |
|---|---|---|---|---|
| CIFAR | 256 | 3/3 | 30.21 | 30.31 |
| CIFAR | 512 | 3/3 | 27.91 | 28.09 |
| CIFAR | 1024 | 3/3 | 26.52 | 26.36 |
| Cats | 1024 | 2/3 | 27.76 | 27.49 |
| Cats | 2048 | 3/3 | 27.96 | 27.44 |
| Cats | 4096 | 3/3 | 26.20 | 25.59 |

CIFAR shows repeatable improvement beyond interpolation. All three widths already interpolate, so this follow-up alone cannot show both descents. Cats has a small validation bump near interpolation, but the averaged test curve falls and the bump is not consistent across seeds. Robust full double descent is not established by these longer runs.

The later improvement is a root-width effect: noisy CIFAR follow-ups use only their ten roots, and the widest Cats models also use only roots. It is not evidence of double descent caused by deeper trees. Axes include allocated parameters and the CNN once; inactive correction positions are not effective inference capacity. Some smaller nodes still have decreasing loss at the budget limit.

# 8. Node visualizations and image sensitivity

We selected roots and correction nodes at different depths, recorded strongly activating images and assigned correction examples, and measured activation frequencies. Grad-CAM [5] targets each node's continuous base score through the frozen CNN. Occlusion masks image regions and checks score changes.

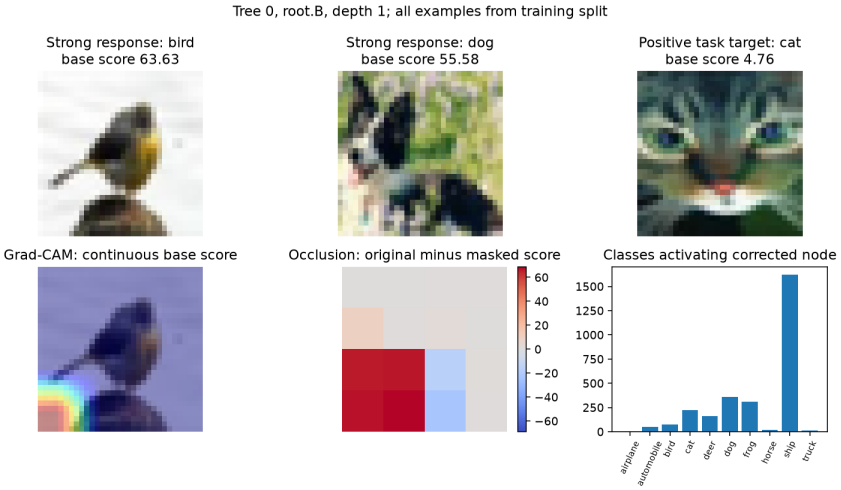



Figure 5. Original CIFAR airplane-tree negative correction: examples, Grad-CAM, occlusion and activation class counts.

The airplane negative child fits seven positive correction targets without errors. Its strongest displayed responses include non-airplane images. The class histogram is dominated by ships; the next page separates broad activations from actual parent errors.

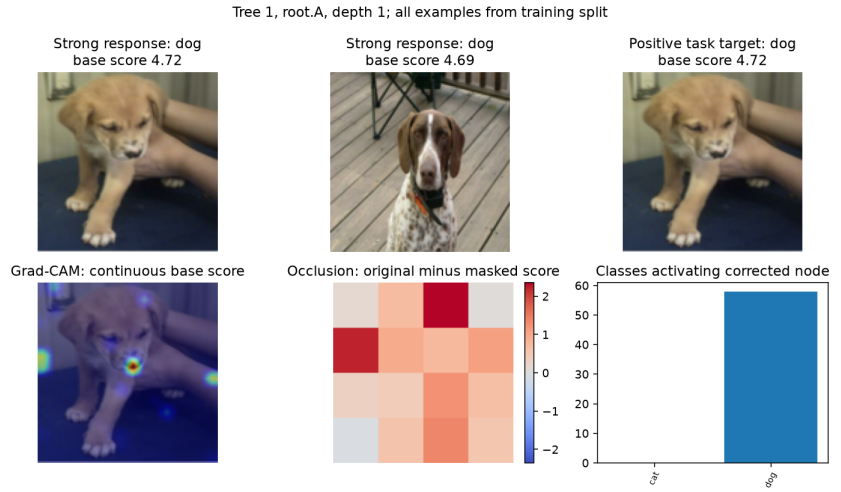



Figure 6. Original Cats positive correction: responses and sensitivity near the animal, with a score-changing occlusion check.

The Cats positive child corrects four positive targets and activates on 58 dogs across the full training split. Its example heatmap emphasizes the muzzle and nearby regions. Saved deeper-node panels also show sensitivity to cages, background and texture.

The shared CNN's final spatial receptive field is 36 x 36 pixels; global pooling gives the node access to the full image. A heatmap is example-specific sensitivity, not proof of a permanent receptive field or a causal object-part detector. Full node panels for both datasets are archived.

# 9. Node observations across depths

Each node has its own assigned correction task, but we also measure its activation over the entire training split. Those are different populations. The table summarizes six saved nodes at depths zero to three.

| Node | Depth | Task size / positives | Full-split activation | Largest activating class |
|---|---|---|---|---|
| CIFAR root | 0 | 45,000 / 4,500 | 10.02% | airplane (4,500) |
| CIFAR root.B | 1 | 4,507 / 7 | 6.29% | ship (1,623) |
| CIFAR root.B.B | 2 | 31 / 24 | 58.09% | frog (3,730) |
| Cats root | 0 | 18,608 / 9,283 | 49.89% | dog (9,283) |
| Cats root.A | 1 | 9,329 / 4 | 0.31% | dog (58) |
| Cats root.A.B.A | 3 | 6 / 2 | 45.50% | cat (4,962) |

All listed correction nodes have zero assigned-task errors. A/B labels describe their sign and their parent task; they are not direct semantic class labels. In the CIFAR row, the root is the airplane binary classifier; in Cats, the root predicts dog.

Ships account for 1,623 of the airplane negative child's activations. However, the root actually makes only seven false positives: three ships, two birds, one cat and one deer. Therefore, 1,623 ship responses do not mean 1,623 airplane/ship mistakes. Most are outside the child's constrained task. Shared blue backgrounds are a possible explanation to investigate, not a conclusion established by these maps.

Depth-2 CIFAR root.B.B fits only 31 assigned examples yet activates on 58.09% of all training images. Depth-3 Cats root.A.B.A fits six examples yet activates on 45.50%. The masks leave most off-task behavior unconstrained. A child response should be interpreted together with the parent score and signed correction, not in isolation.

The shallow Cats positive child is much more selective: 0.31% activation, all on dogs. Its puppy example suggests sensitivity to unusual appearances. Deeper caged-cat examples emphasize cage/background regions, so both animal appearance and context can affect scores. These are observations on saved examples, not proofs of stable object-part detectors.

All V1 nodes access the full image through a common frozen trunk, but their MLPs weight those features differently. Grad-CAM can show different example-specific sensitivities even with shared features. V2 pixel nodes also connect to the full image: reshaped first-layer weights would show spatial weighting, not automatically localized receptive fields.

# 10. Smaller models, effort and conclusions

We removed correction subtrees with a cumulative validation tolerance of 0.5 percentage points and retrained narrower nodes at widths 32 and 16. Training fit was checked again. All totals include the shared CNN once.

| Original-label model | Parameters | Nodes | Train % | Test % |
|---|---|---|---|---|
| CIFAR / selected | 486,784 | 24 | 99.9933 | 80.02 |
| CIFAR / pruned | 370,248 | 10 | 99.9711 | 80.01 |
| CIFAR / smallest near-fit | 316,184 | 14 | 99.9933 | 79.70 |
| Cats/Dogs / selected | 336,952 | 6 | 100.0000 | 89.94 |
| Cats/Dogs / pruned | 295,332 | 1 | 99.9678 | 89.94 |
| Cats/Dogs / smallest near-fit | 292,228 | 5 | 99.9678 | 89.94 |

Pruning reduced CIFAR parameters by 23.94% and Cats parameters by 12.35%. The smallest tested near-fit models use 316,184 and 292,228 parameters respectively. These are the smallest found in our experiments, not global minima; narrower nodes can require more correction children.

Pruning removes every V1 correction child: CIFAR retains ten roots, and Cats retains one. Training accuracy remains near-perfect, with virtually unchanged measured validation/test accuracy. The children improved training fit but were unnecessary under our validation tolerance; this does not prove that all correction trees only memorize.

The shared trunk has 287,008 parameters: about 78% of pruned CIFAR and 97% of pruned Cats. We reduced the tree, not the trunk. Larger future savings would require shrinking CNN channels, distillation or quantization; these were not tested.

Effort includes four feature strategies, fixed-root and matched-classifier controls, width/depth/epoch studies, 64 noise/control configurations and 48 longer-budget configurations, a paired fitting repair, interpretation and compression. Continued runs are not counted as independent new samples.

The main innovations in this submission are the deliberate feature/root restrictions that expose the tree contribution and the targeted parent-fitting refinement tested with unchanged bases. Established CNNs, MLPs, Grad-CAM and pruning support the implementation. We kept negative results because they explain the next experiment.

We achieved near-perfect fit on both datasets, exposed when corrections help, found suggestive double-descent shapes and repeatable later improvement, and reduced model size. Perfect fitting did not guarantee stronger generalization.

Future work should vary depth while keeping the root and features fixed to test tree-driven double descent, repeat with independent feature/noise seeds and a fresh holdout, and compare against a CNN trained directly on noisy labels. Trunk compression and a shared multiclass correction structure could reduce deployment size.

# Bibliography and glossary

[1] Jayadeva. A Tree Type Neural Network. Supplied teaching manuscript; correction targets from Table II.

[2] Krizhevsky, A. Learning Multiple Layers of Features from Tiny Images (2009); CIFAR-10 dataset. Source

[3] TensorFlow Datasets. Cats vs Dogs dataset catalog. Source

[4] Nakkiran et al. Deep Double Descent: Where Bigger Models and More Data Hurt (2019). Source

[5] Selvaraju et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization (2017). Source

# Short glossary

| Term | Meaning in this report |
|---|---|
| CNN / trunk | Convolutional neural network that turns an image into features. |
| MLP / node | Small fully connected network that produces a scalar score. |
| LP / margin | Linear program fitting correction weights / signed score distance from the decision boundary. |
| Width / depth | Hidden units in a node / number of correction levels below the root. |
| Epoch | One pass through a node's assigned training examples. |
| Overfitting | Fitting training examples very well while unseen performance remains weaker. |
| Interpolation | Zero classification errors on the designated training split. |
| Double descent | Held-out error falls, rises near interpolation, then falls again as capacity increases. |
| Parameters | Network weights and biases, including frozen ones; the shared extractor is counted once. |
| Allocated / active | All reserved weights / weights on paths with nonzero correction contributions. |
| One-versus-rest | One binary classifier per class; CIFAR chooses the largest corrected score. |
| Label noise | Replacing selected training labels with incorrect ones. Evaluation labels stay unchanged. |
| Grad-CAM / occlusion | Gradient sensitivity map / measuring score changes after masking an image region. |

Repository contains runnable code, saved measurements, split manifests, detailed figures and four selected deployment models. The ZIPs are organizational only: negative results and failed fitting attempts remain in the evidence. Grant instructor access and submit through the course process.

Repository: https://github.com/Harshal-Narsaria/ELL409